LAB 3 : FILE ALLOCATION STRATEGIES (SEQUENTIAL)

OCC : 1 GROUP : 8

INTRODUCTION TO SEQUENTIAL FILE ALLOCATION

Sequential file allocation is a strategy where each file occupies a set of contiguous blocks on the disk. This method ensures that disk blocks are allocated in a linear sequence.

Formula

if a file starts at block b and has a length of n blocks, it occupies blocks b, b+1, ..., b+(n−1).b, b+1, ..., b+n−1.

Primary use

simulate the way data is stored and accessed as a physical entity on a disk.

KEY CHARACTERISTICS

Simple Data Structure
Only the starting block address and the length of the file are required to manage the allocation.

External Fragmentation
As files are created and deleted, free disk space is broken into small chunks. A new file may not be allocated if a single contiguous hole of sufficient size is unavailable, even if the total free space is enough.

High Performance for Sequential Access
Since the blocks are adjacent, the disk head movement is minimized during read/write operations, making it highly efficient for sequential data processing.

Difficulty in File Growth
If a file needs to expand beyond its initial length, it may be blocked by an adjacent allocated file, requiring the entire file to be moved to a larger hole.

WORKING PRINCIPLE

01

Allocation Criteria
When a file is created, the system searches for a contiguous set of free blocks that matches the requested length.

02

Mapping
To access the i^{th} block of a file starting at block b, the operating system simply accesses block b + i.

03

Directory Entry
The directory typically stores the file name, the starting block address, and the total length of the file.

CODE IMPLEMENTATION

```
import java.util.Scanner;

public class Main
{
public static void main(String[] args) {
int[] disk = new int[50];
for(int i = 0 ; i < disk.length ; i++){
 disk[i] = 0; //set all index to 0, not occupied
}
 System.out.println("Sequential File Allocation:\n");
 recurse(disk);

}
```

CODE IMPLEMENTATION

```
public static void recurse(int disk[]){
 Scanner sc = new Scanner(System.in);
 int flag = 0,startBlock,length,j,k,ch;
 System.out.println("Enter the starting block:");
 startBlock = sc.nextInt();
 System.out.println("Enter the length of the files:");
 length=sc.nextInt();

 if((startBlock+length)>= disk.length){
 System.out.println("The file is not allocated in the disk\n");
 }

 for(j=startBlock;j<(startBlock+length);j++){
 if(disk[j]==0){
 flag++; //make sure all is 0
 }
 }
```

CODE IMPLEMENTATION

```
System.out.println();

 if(length==flag){
 for(k=startBlock;k<(startBlock+length);k++){
 if(disk[k]==0){
 disk[k]=1;
 System.out.print(k);

 if(k<(startBlock+length)-1){
 System.out.print(" | ");
 }
 }
 }
 if(k!=(startBlock+length-1)){
 System.out.println("\n\nThe file is allocated to the disk\n");
 }
```

CODE IMPLEMENTATION

```
}
else {
 System.out.println("The file is not allocated in the disk\n");
}
System.out.println("Do you want to enter more files?\n");
System.out.println("Enter 1 for YES, 0 for NO: ");
ch=sc.nextInt();

if(ch==1){
 recurse(disk);

}
else{
 return;
}
}
}
```

PROGRAM SAMPLE OUTPUT

```
Sequential File Allocation:

Enter the starting block:
3
Enter the length of the files:
20

3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22

The file is allocated to the disk

Do you want to enter more files?

Enter 1 for YES, 0 for NO:
1
Enter the starting block:
20
Enter the length of the files:
5

The file is not allocated in the disk

Do you want to enter more files?

Enter 1 for YES, 0 for NO:
0
```

THANKYOU